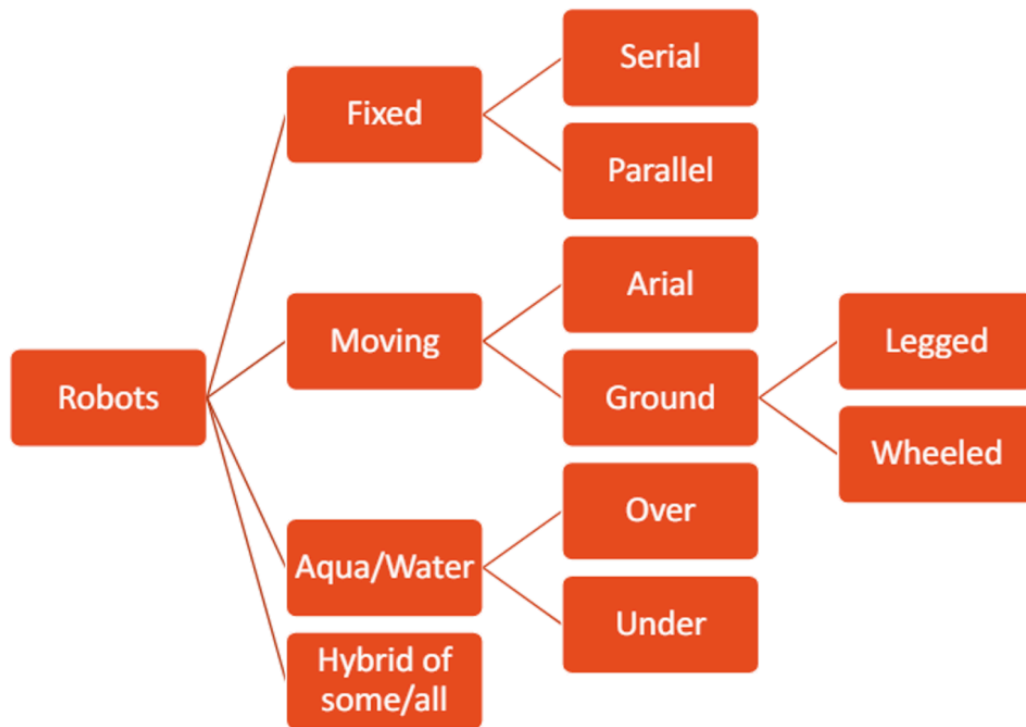
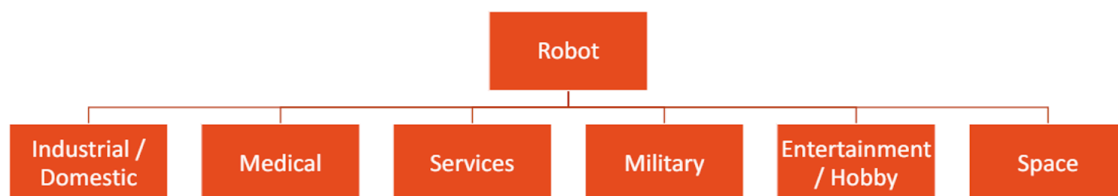




동작 방식에 따른 분류



사용 용도에 따른 분류

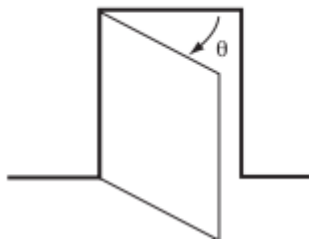
로봇의 구성요소

기구부 : 로봇 프레임, 하우징, 바퀴 및 바퀴 축, 베어링, 기어, 벨트, 말단효과장치

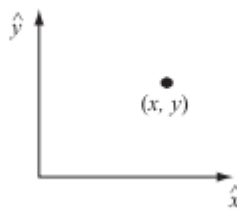
하드웨어부 : 컴퓨터, 제어기, 전원 과리 보드, 구동기(모터), 센서

소프트웨어부 : 하드웨어 제어 프로그램(모터 제어 프로그램, 센서 입출력 프로그램), 운영체제, 각종 응용 소프트웨어 등

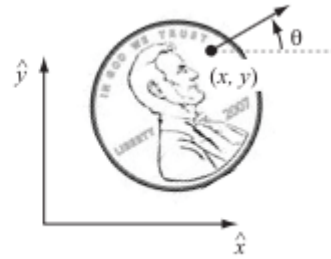
자유도 : 관절에서 허용되는 독립적인 움직임, 방향의 수



자유도 1개

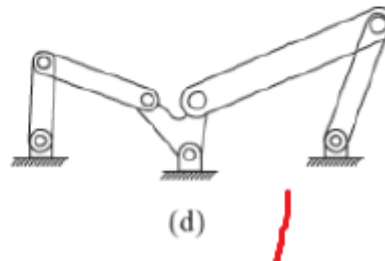
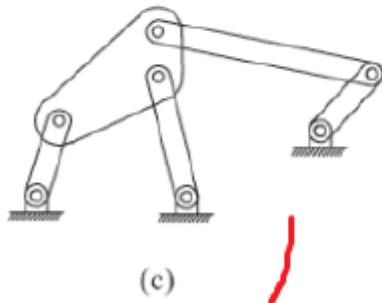
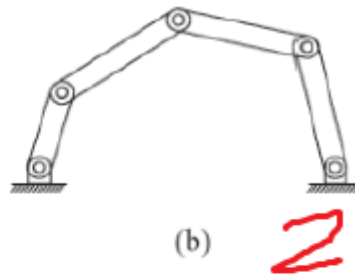
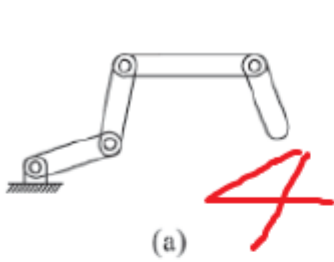


자유도 2개



자유도 3개

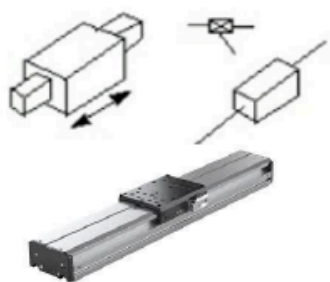
Degrees of freedom = (number of variables) - (number of independent equations)



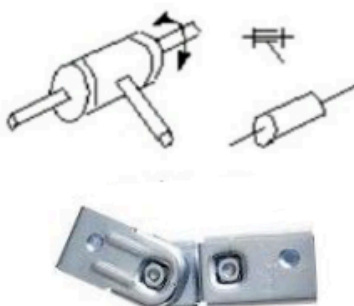
Gruebler의 공식

$$dof = 3(N - 1 - J) + \sum_{i=1}^J f_i$$

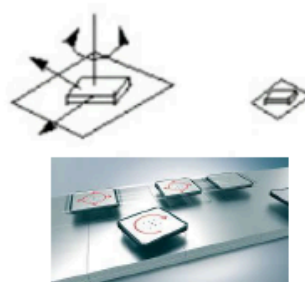
- **M** = 기계의 자유도(모빌리티).
- **n** = 링크(링크를 포함한 고정 베이스 포함)의 수.
- **j** = 조인트(혹은 조인트 쌍)의 수.
- **r** = 중복된 제약 조건의 수(독립적이지 않은 제약 조건).



리니어 모터



9자 조인트



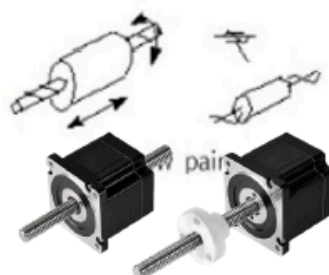
플라나 모터



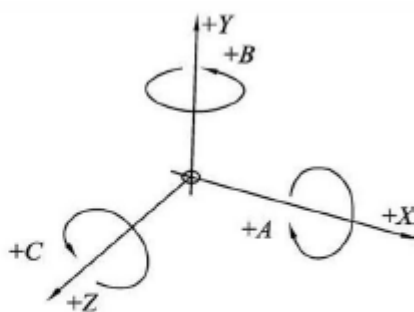
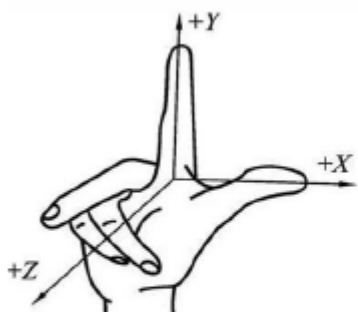
삼각대 볼조인트



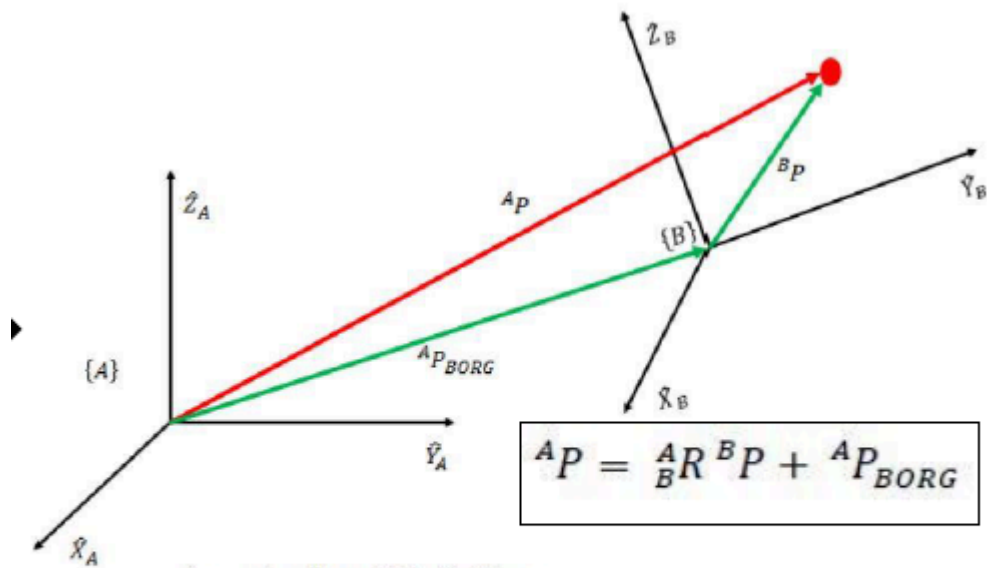
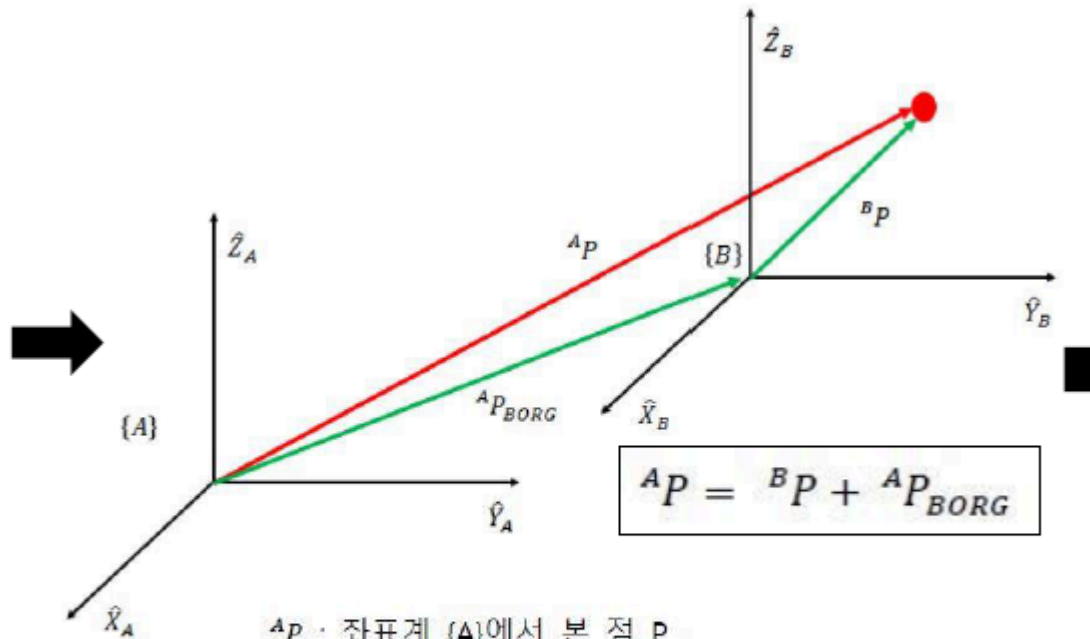
공압 액추에이터

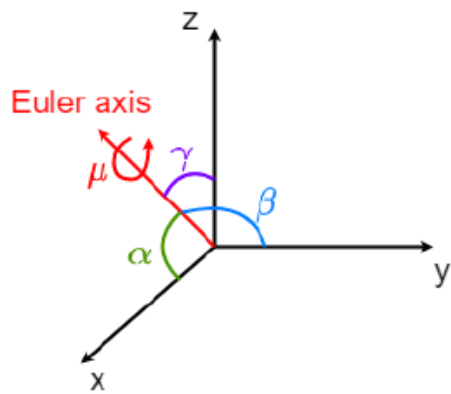


리니어 액추에이터

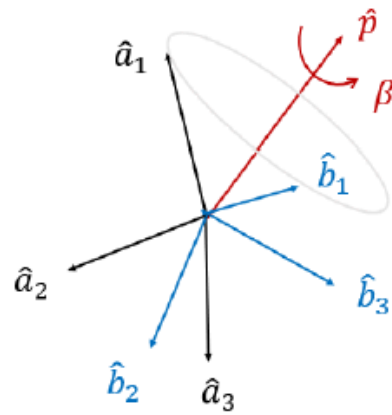


오른손 좌표계





카르테시안(Cartesian) 공간

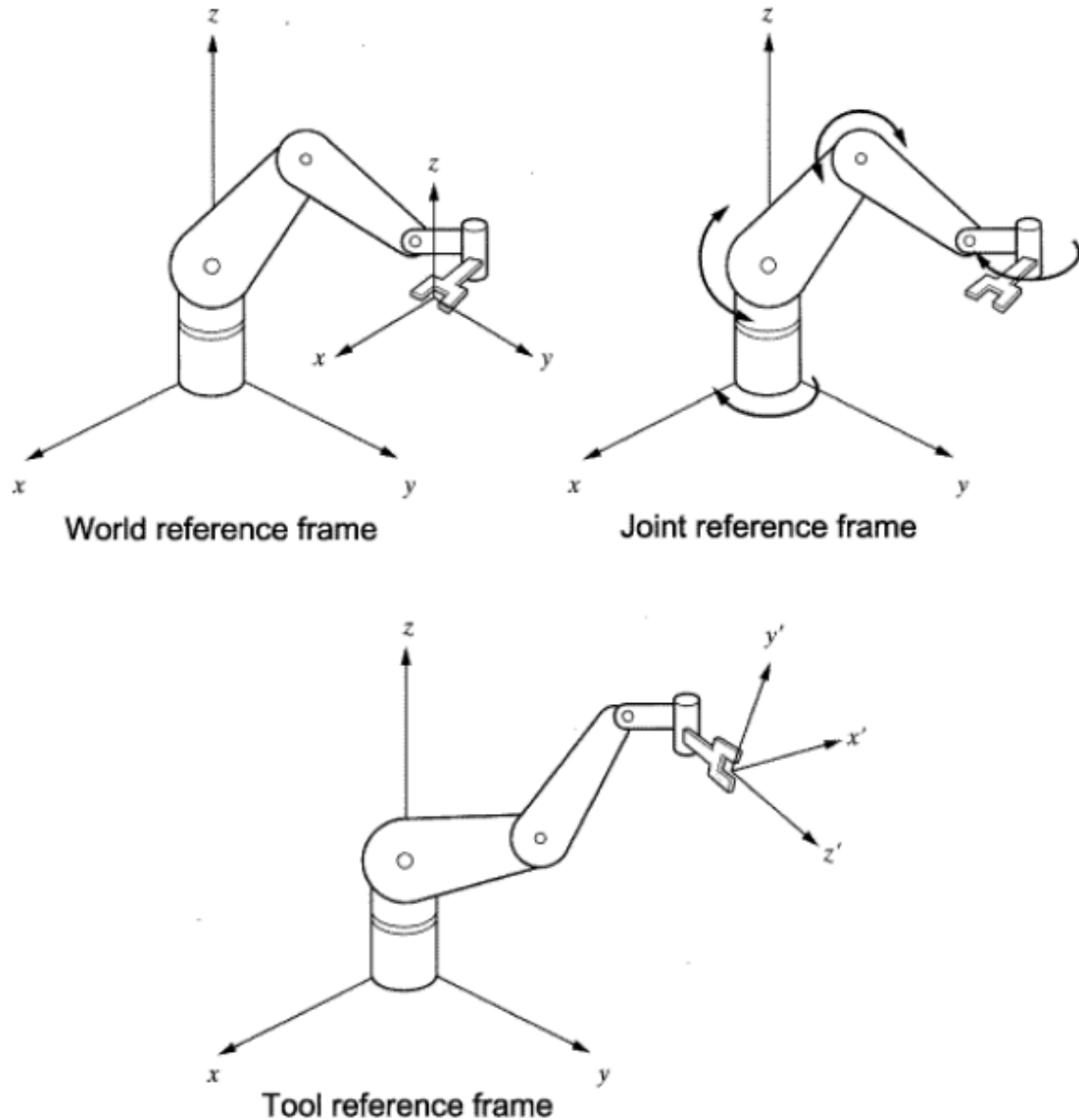


쿼터니언(Quaternion) 공간

World Reference Frame: 로봇 작업 공간의 기준이 되는 고정된 좌표계.

Joint Reference Frame: 각 조인트의 이동과 회전을 기준으로 하는 좌표계.

Tool Reference Frame: 로봇의 툴 또는 엔드 이펙터의 위치와 방향을 기준으로 하는 좌표계.



로봇기구학

순기구학 : 조인트 공간에서 카르테시안 공간 표현, 로봇의 관절이 특정 각도로 설정되어 있을 때, 이 관절을 이용해 로봇의 손 끝이 작업 공간에서 어떤 위치와 방향에 있는지 계산

역기구학 : 로봇을 원하는 위치로 동작시킬 경우, 각 관절에서 어떠한 각도를 가지고 있어야 하는지 계산 (기하학적 방법, 대수적 방법, 수치해석적 방법)

Singularity

매니퓰레이터가 작업을 할 수 있는 영역은 매니퓰레이터가 도달 할 수 있는 영역을 뜻하며, 이를 작업공간 혹은 작업영역

D-H(Denavit & Hartenberg) Parameter

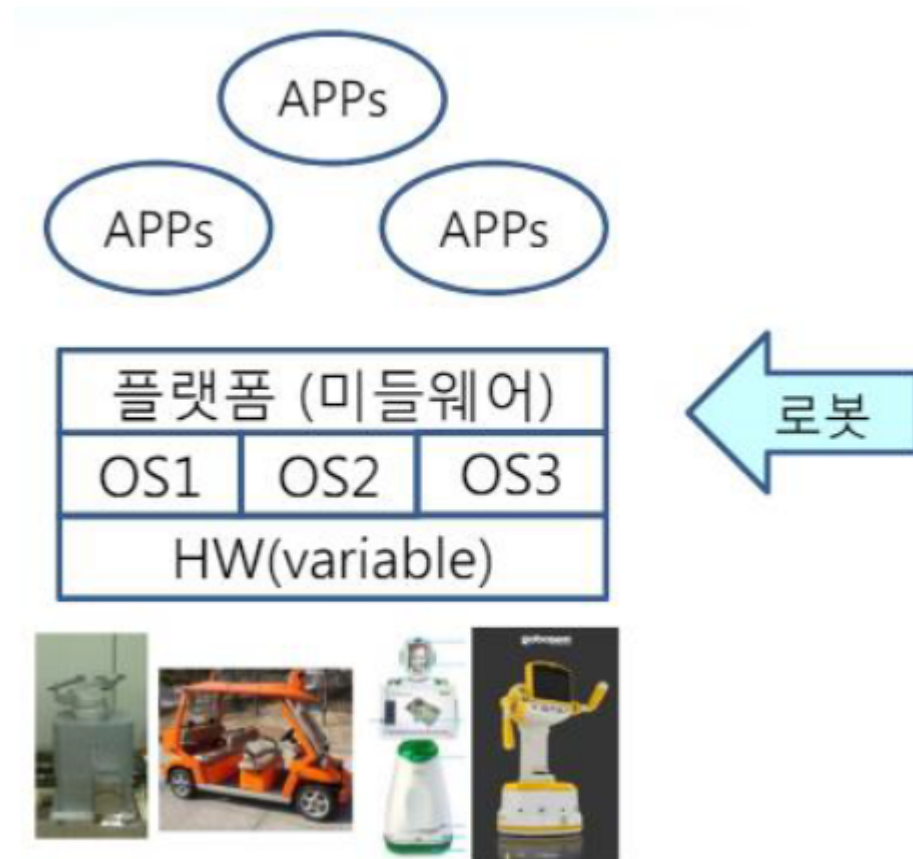
로봇의 모델링하는 방법 Manipulator의 Link와 Joint간의 관계를 수식적으로 표현

로봇 SW

하드웨어 인터페이스 통합

하드웨어 추상화 규격화 모듈화

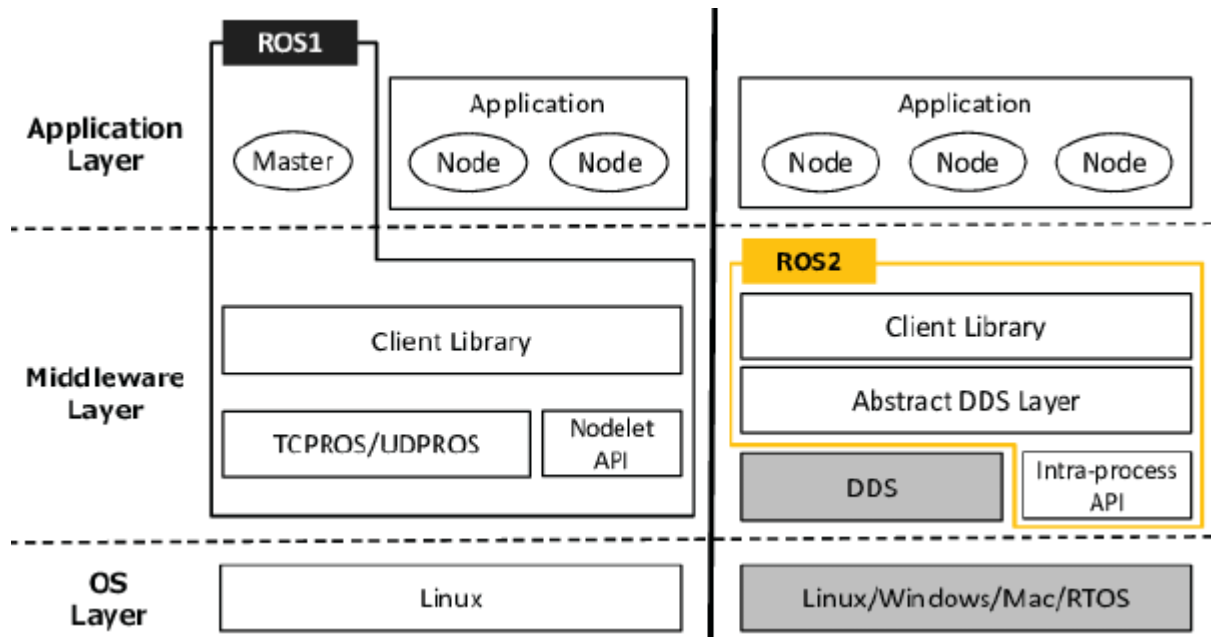
하드웨어 운영체제 애플리케이션 분리



ROS 로봇을 위한 오픈 소스 메타 운영 시스템

하드웨어 추상화, 하위 디바이스 제어

메타 운영 체제 : 애플리케이션과 분산 컴퓨팅 자원간의 가상화 레이어로 분산 컴퓨팅 자원을 활용하여 스케줄링 및 로드, 감시, 에러 처리 등 실행하는 시스템



ROS	ROS2
단일 로봇	복수대의 로봇
워크스테이션급 컴퓨터	임베디드 시스템에서의 ROS 사용
Linux 환경, 실시간 제어 지원하지 않음	Linux 환경에서의 실시간 제어
안정된 네트워크 환경 요구	불안정한 네트워크 환경에서도 동작할 수 있는 유연함
단일 플랫폼(Linux)	멀티 플랫폼(Linux, Windows, MacOS)
	최신 기술 지원(Zeroconf, Protocol Buffers, ZeroMQ, WebSockets, DDS 등)
주로 대학이나 연구소 등의 아카데미 연구 용도	상업용 제품 지원

```

Python
colcon build

# Turtlesim 설치
sudo apt update
sudo apt install ros-humble turtlesim

ros2 run turtlesim turtlesim_node
ros2 run turtlesim turtle_teleop_key

ros2 pkg list
ros2 node list
ros2 topic list
ros2 service list

```



```
ros2 action list
rqt_graph
```

토픽

비동기식 단방향 메시지 송수신

msg 인터페이스 **Publisher - Subscriber**

서비스

동기식 양방향 메시지 송수신

서비스 요청 **Request Server Client**

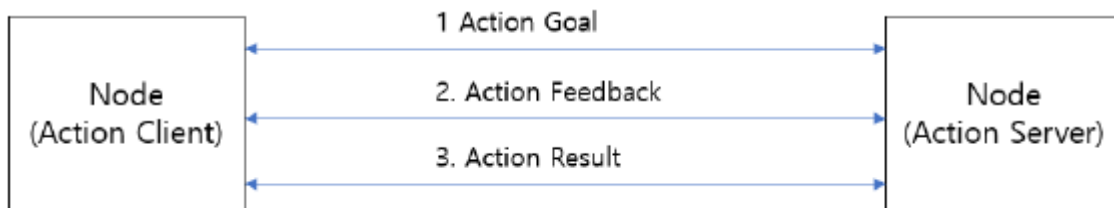
요청 받은 서비스 수행 **Response Server Server**

액션

비동기식, 동기식 양방향 메시지 송수신

액션 목표 지정 **Action Client**

액션 목표(**Goal**) 태스크 수행 - 중간 결과값 전송 액션(**Feedback**), 최종 결과값 액션
결과(**Result**) 전송 **Action Server**



파라미터 노드 설정 값

```
Python
gedit ~/.bashrc

"""
source /opt/ros/humble/setup.bash
alias humble='source /opt/ros/humble/setup.bash; echo \"ROS2 humble is
activated/\"'
alias sb='source ~/.bashrc
"""

source ~/.bashrc

ros2 run turtlesim turtlesim_node
ros2 run launch turtlesim multisim.launch.py

ros2 node list
ros2 node info /turtlesim
```

```
ros2 topic list

ros2 topic info /turtle1/pose

ros2 topic type

ros2 topic echo

ros2 topic pub --once turtle1/cmd_vel geometry_msgs/msg/Twist "linear: {x:
2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}"

ros2 topic bw

ros2 topic delay

ros2 topic find

ros2 topic hz

ros2 service list

ros2 service type

ros2 service call /turtle1/set_pen

ros2 service find turtlesim/srv/TeleportAbsolute

ros2 service call /turtle1/teleport_absolute turtlesim/srv/TeleportAbsolute
"{x: 2, y: 2, theta: 1.57}"

ros2 action list

ros2 action info /turtle1/rotate_absolute

ros2 action sent_goal
/turtle1/rotate_absolute_turtlesim/action/RotateAbsolute "{theta: 3.14}"

ros2 param list

ros2 param describe /teleop_turtle_scale_angular

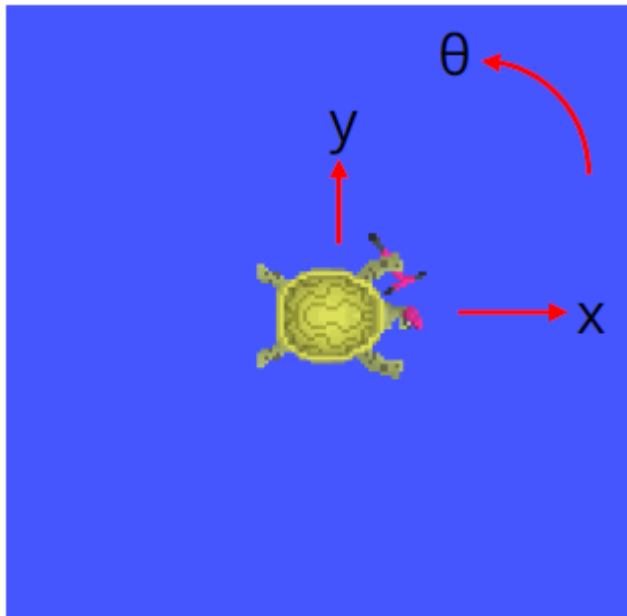
ros2 interface show

ros2 interface list

ros2 interface package
```

```
ros2 interface packages
```

```
ros2 interface proto
```



Turtlesim의 자세 $x, y, \theta(\text{radian})$

Degrees $\rightarrow$ radians $\times \text{ by } \frac{\pi}{180}$ Radians $\rightarrow$ degrees $\times \text{ by } \frac{180}{\pi}$

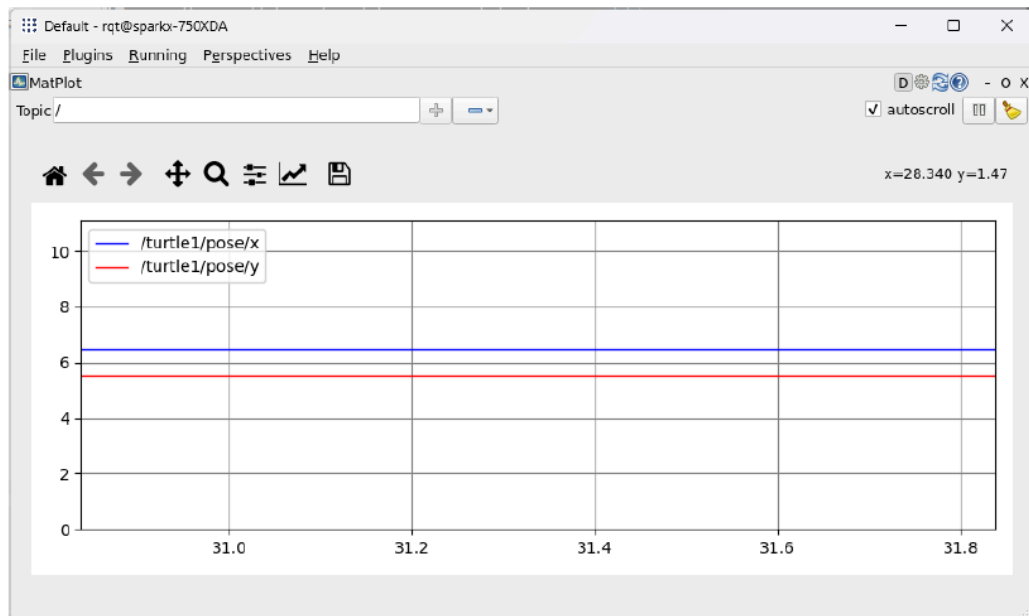
```
Python
```

```
# rqt
```

```
ros2 run rqt_msg rqt_msg
```

Rqt Plot

- Plugins -> Visualization -> Plot



- Topic Monitor
 - Plugins -> Topic -> Topic Monitor

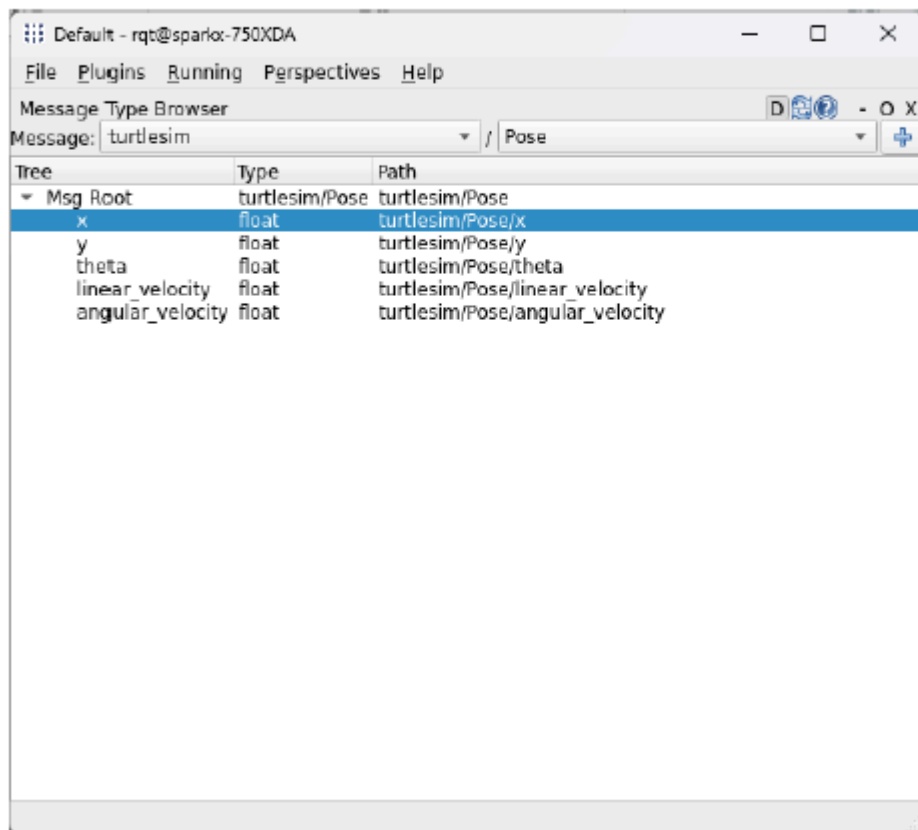
The screenshot shows the Rqt Topic Monitor window with the following details:

- Window Title:** Default - rqt@sparkx-750XDA
- Menu Bar:** File, Plugins, Running, Perspectives, Help
- Table:** A table with 5 columns: Topic, Type, Bandwidth, Hz, and Value.

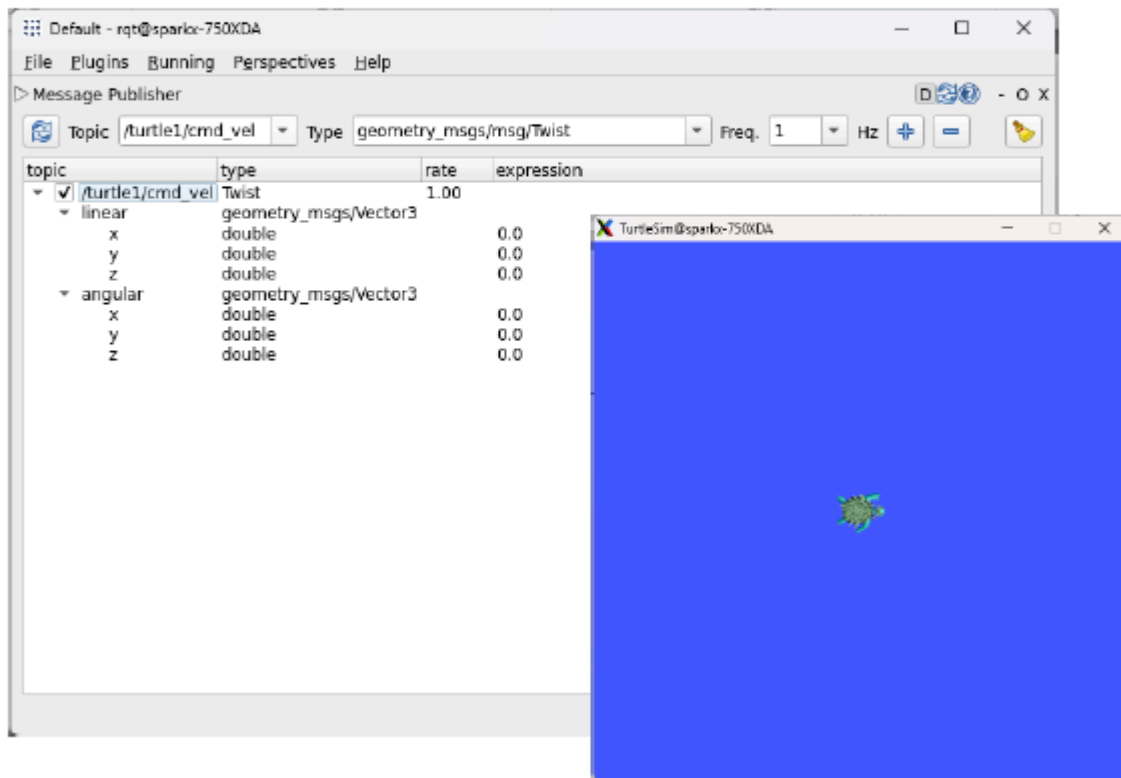
Topic	Type	Bandwidth	Hz	Value
☐ /parameter_events	rcl_interfaces/msg/ParameterEvent			not monitored
☐ /rosout	rcl_interfaces/msg/Log			not monitored
☒ /turtle1/cmd_vel	geometry_msgs/msg/Twist	unknown	unknown	
☐ linear	geometry_msgs/Vector3			
☐ angular	geometry_msgs/Vector3			
☒ /turtle1/color_sensor	turtlesim/msg/Color	unknown	62.40	
☐ r	uint8			179
☐ g	uint8			184
☐ b	uint8			255
☒ /turtle1/pose	turtlesim/msg/Pose	unknown	62.40	
☐ x	float			6.504444599...
☐ y	float			5.544444561...
☐ theta	float			-2.01600003...
☐ linear_velocity	float			0.0
☐ angular_velocity	float			0.0
☐ /turtle1/rotate_absolute/_action/feedback	turtlesim/action/RotateAbsolute_FeedbackMessage			can not get ...
☒ /turtle1/rotate_absolute/_action/status	action_msgs/msg/GoalStatusArray	unknown	unknown	
☐ status_list	sequence<action_msgs/GoalStatus>			

Message Type Browser

- Plugins -> Topic -> Message Type Browser

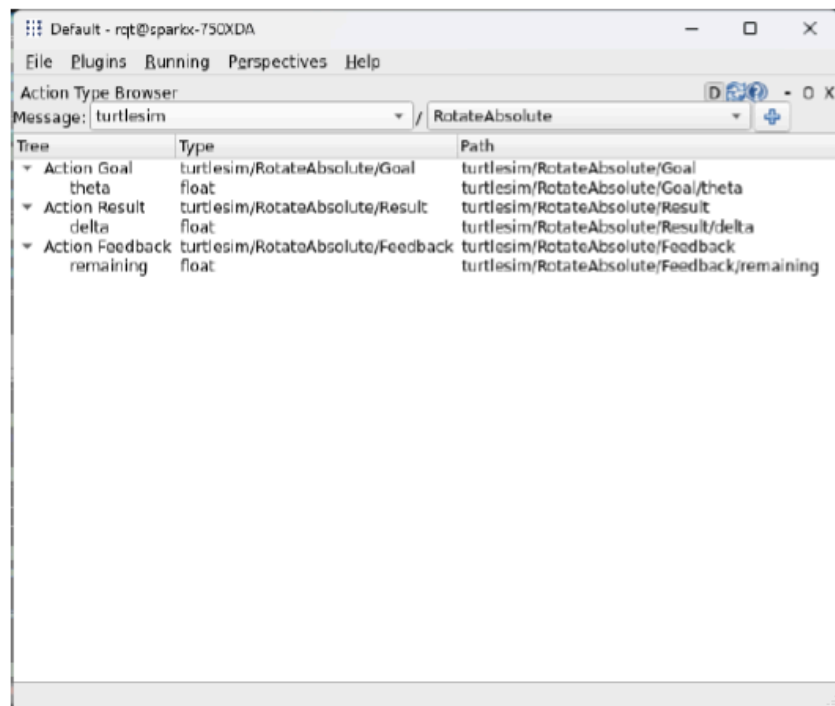


- Topic Publisher
 - Plugins -> Topic -> Topic Publisher



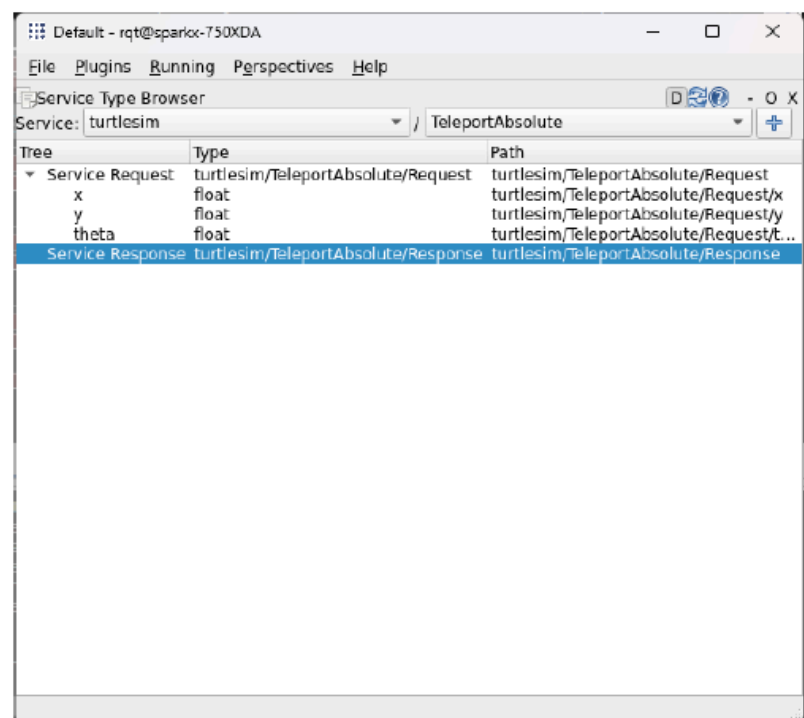
Action Type Brower

- Plugins -> Action -> Action Type Browser

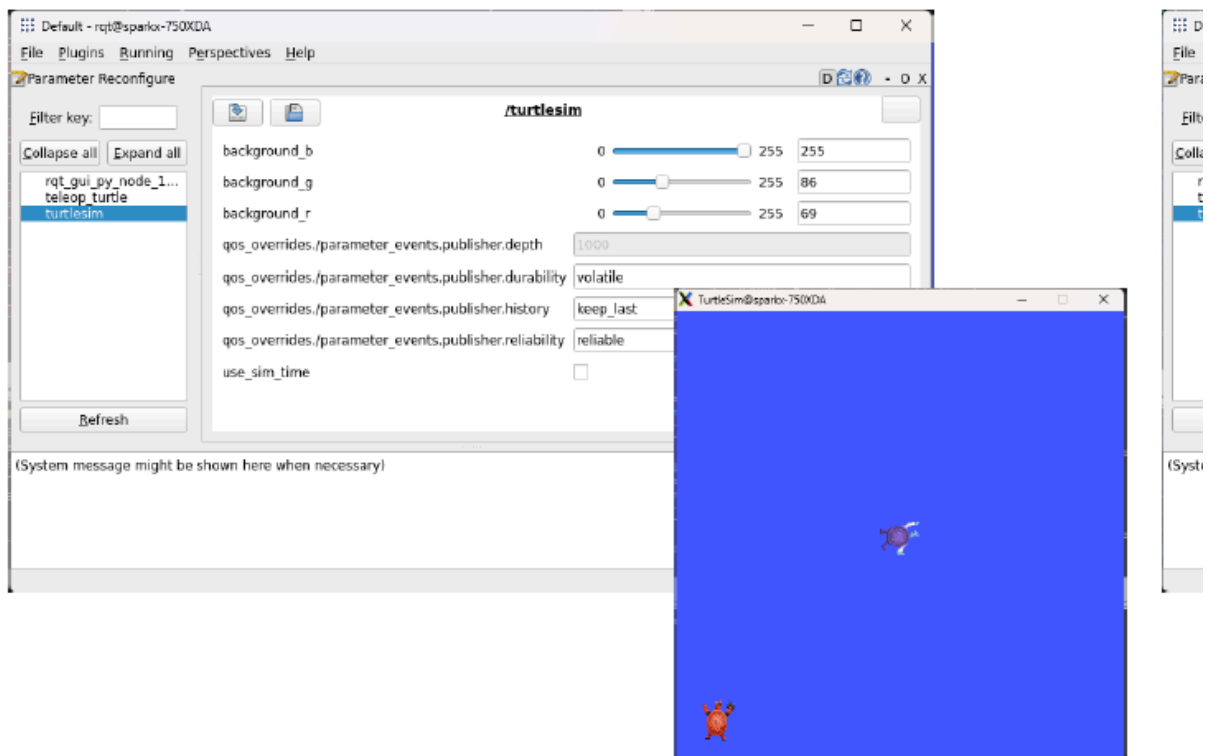


Service Type Browser

- Plugins -> Service -> Service Type Browser



- Parameter Reconfigure
 - Plugins -> Configuration -> Dynamic Reconfigure



CMake

```
Python
mkdir -p ros2_ws/src

cd ros2_ws/src

ros2 pkg create --build-type ament_cmake --license Apache-2.0 my_package
```

1. 패키지 생성

```
mkdir ros2_ws/src
```

```
cd ros2_ws/src
```

```
ros2 pkg create --build-type ament_python py_pubsub
```

```
ls
```


tree

```
cd ~/ros2_ws/src/py_pubsub/py_pubsub
```

wget

```
https://raw.githubusercontent.com/ros2/examples/humble/rclpy/topics/minimal_publisher/examples_rclpy_minimal_publisher/publisher_member_function.py
```

ls

```
cd ~/ros2_ws
```

```
rosdep install -i --from-path src --rosdistro humble -y
```

```
colcon build --packages-select py_pubsub
```

```
source install/setup.bash
```

```
ros2 run py_pubsub talker
```

새 터미널 오픈

humble

```
cd ros2_ws
```

```
source install/setup.bash
```

```
ros2 run py_pubsub listener
```

```
source usr/share/colcon_cd/function/colcon_cd.sh
```

```
export _colcon_cd=~/.ros2_ws
```

```
colcon_cd my_package
```

2. namespace

```
ros2 run turtlesim turtlesim_node
```

```
rqt_graph
```

```
ros2 run turtlesim turtlesim_node -ros-args -r __ns:=/robot1
```

```
ros2 run turtlesim turtlesim_node -ros-args -r __ns:=/robot2
```

rqt_graph

3. 인터페이스 패키지 생성

```
cd ros2_ws/src
```

```
ros2 pkg create --build-type ament_cmake ros_study_msgs
```

```
cd ros_study_msgs/
```

```
tree
```

```
cd ../../
```

```
colcon build --packages-select ros_study_msgs
```

```
ls
```

```
tree
```

```
colcon build --packages-select ros_study_py
```

```
source install/setup.bash
```

```
ros2 run ros_study_py my_msg_test
```

```
ros2 topic list
```

```
rosdep install --from-paths src --ignore-src -r -y
```

```
colcon build --packages-select ros_study_msgs ex_calculator
```

```
ros2 run ex_calculator calculator
```

```
ros2 run ex_calculator argument
```

```
ros2 run ex_calculator operator
```

```
ros2 run ex_calculator checker
```

Type name	C++	Python	DDS type
bool	bool	builtins.bool	boolean
byte	uint8_t	builtins.bytes*	octet
char	char	builtins.str*	char
float32	float	builtins.float*	float
float64	double	builtins.float*	double
int8	int8_t	builtins.int*	octet
uint8	uint8_t	builtins.int*	octet
int16	int16_t	builtins.int*	short
uint16	uint16_t	builtins.int*	unsigned short
int32	int32_t	builtins.int*	long
uint32	uint32_t	builtins.int*	unsigned long
int64	int64_t	builtins.int*	long long
uint64	uint64_t	builtins.int*	unsigned long long
string	std::string	builtins.str	string
wstring	std::u16string	builtins.str	wstring

Type name	C++	Python	DDS type
static array	std::array<T, N>	builtins.list*	T[N]
unbounded dynamic array	std::vector	builtins.list	sequence
bounded dynamic array	custom_class<T, N>	builtins.list*	sequence<T, N>
bounded string	std::string	builtins.str*	string

	토픽	서비스	액션
연속성	연속성	일회성	복합(토픽 + 서비스)
방향성	단방향	양방향	양방향
동기성	비동기	동기	동기 + 비동기
다자간연결	1:1, 1:N, N:1, N:N	1:1	1:1
노드역할	Publisher, Subscriber	Server, Client	Server, Client
동작트리거	퍼블리셔	클라이언트	action 인터페이스
인터페이스	msg	srv	action
CLI명령어	ros2 topic	ros2 service	ros2 action
사용 예	센서 데이터, 로봇 상태, 로봇 좌표, 로봇 속도 명령 등	LED 제어, 모터 토크 On/Off IK/FK 계산, 이동 경로 계산 등	목적지로 이동, 물건 파지, 복합 테스트

우분투 명령어

ls: 디렉토리 내 파일 및 폴더 목록을 표시합니다.

cd: 디렉토리를 변경합니다.

pwd: 현재 작업 중인 디렉토리의 절대 경로를 출력합니다.

cp: 파일이나 디렉토리를 복사합니다.

mv: 파일이나 디렉토리를 이동하거나 이름을 변경합니다.

rm: 파일이나 디렉토리를 삭제합니다.

mkdir: 새 디렉토리를 생성합니다.

rmdir: 빈 디렉토리를 삭제합니다.

touch: 빈 파일을 생성하거나 파일의 수정 시간을 업데이트합니다.

cat: 파일의 내용을 출력합니다.

grep: 파일에서 패턴을 검색합니다.

find: 파일 및 디렉토리를 검색합니다.

chmod: 파일이나 디렉토리의 권한을 변경합니다.

sudo: 슈퍼유저 권한으로 명령어를 실행합니다.

apt: 패키지를 설치, 업데이트, 제거합니다.

df: 파일 시스템의 디스크 공간 사용량을 출력합니다.

du: 파일과 디렉토리의 디스크 사용량을 확인합니다.

ps: 현재 실행 중인 프로세스 목록을 표시합니다.

top: 시스템의 실시간 프로세스 및 자원 사용량을 모니터링합니다.

kill: 프로세스를 종료합니다.

파일 생성 명령어

touch: 빈 파일을 생성하거나 파일의 수정 시간을 업데이트합니다.

echo: 문자열을 파일에 작성하여 새 파일을 생성합니다.

printf: 포맷된 문자열을 파일에 작성하여 새 파일을 생성합니다.

cat: 표준 입력으로부터 파일을 생성합니다.

cp: 빈 파일을 복사하여 새 파일을 생성합니다.

nano: 터미널 기반의 간단한 텍스트 편집기입니다.

gedit: GNOME 데스크탑 환경용 그래픽 텍스트 편집기입니다.

ros2 패키지

C

- `CMakeLists.txt` 패키지 내에서 코드를 빌드하는 방법을 설명하는 파일
- `include/<package_name>` 패키지의 공개 헤더를 포함하는 디렉토리
- `package.xml` 패키지에 대한 메타 정보를 포함하는 파일
- `src` 패키지의 소스 코드가 포함된 디렉토리

C/C++

```
my_package/  
  CMakeLists.txt  
  include/my_package/  
  package.xml  
  src/
```

Python

- `package.xml` 패키지에 대한 메타 정보를 포함하는 파일
- `resource/<package_name>` 패키지에 대한 마커 파일
- `setup.cfg` 패키지에 실행 파일이 있는 경우 필요하므로 찾을 수 있습니다.`ros2 run`
- `setup.py` 패키지 설치 방법에 대한 지침이 포함되어 있습니다
- `<package_name>`- ROS 2 도구에서 패키지를 찾는 데 사용되는 패키지과 동일한 이름의 디렉토리에는 다음이 포함됩니다.`__init__.py`

Python

```
my_package/  
    package.xml  
    resource/my_package  
    setup.cfg  
    setup.py  
    my_package/
```